



Embedding Flaris in a C application

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.2.1 **Generated** 2026-09-07 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Flaris ships as a library as well as a program. Link `libflaris` into your application and Flaris becomes its scripting layer: load scripts, call their functions from C, and drive the scheduler from your own loop.

This is the counterpart to the [FFI guide](#). FFI is *a script reaching a shared library you did not compile against*; this is *your application and a script calling each other directly*, in the same process, with nothing copied between them.

- 1. What you download
- 2. Hello, host
- 3. Shipping scripts with your application
- 4. Lifecycle
- 5. Configuration
- 6. Loading code
- 7. Calling into Flaris
- 8. Values and ownership
- 9. Native modules: your C functions, called from script
- 10. Restricting what a script can reach
 - 10a. Signals from the script
 - 10b. Embedding from C#
- 11. Driving the scheduler
- 12. Errors and diagnostics
- 13. Limits and what is not supported
- 14. API reference

1. What you download

Get the library for your platform from the [downloads page](#). Each archive unpacks to the same shape:

```
flaris-lib/  
├─ include/  
│   └─ flaris.h      # the embedding API – the only header you include  
└─ lib/              # the static and shared library
```

One header, and it is self-contained: it declares the whole API in terms of C types, so nothing about the VM's internals leaks into your build. Values cross the boundary as `FlarisValue`, an opaque one-word handle.

Platform	Static	Shared
macOS ARM64	<code>libflaris.a</code>	<code>libflaris.dylib</code>

Platform	Static	Shared
Linux x86_64 / ARM64	libflaris.a	libflaris.so
Windows x64	libflaris.a	flaris.dll + libflaris.dll.a

This is the same VM the `flarism` command runs — compiler, runtime, JIT, standard builtins and debugger — without the command-line front end. Nothing a script does, and nothing a bad input file contains, makes it call `exit()`; your application stays in control. Section 12 states the guarantee exactly.

Include path and link flags. Point your compiler at `include/`, link the library, and add the system libraries the VM needs:

```
# macOS
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -framework Security -framework CoreFoundation

# Linux
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -ldl -lpthread

# Windows (MSYS2 clang64)
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lws2_32 -lbcrypt -liphlpapi -lsecur32 -lbcrypt32
```

Linking the shared library instead. The shared library exports exactly what this document describes - the Flaris* functions listed in section 13, plus `flarisConfigDefaults` - and nothing else; the VM's internals are hidden, so no future release can break you by moving one. On Windows, add `-DFLARIS_DLL` when you build against `flaris.dll`, so the declarations become `dllimport`:

```
cc host.c -DFLARIS_DLL flaris-lib/lib/libflaris.dll.a -Iflaris-lib/include -o host
```

macOS and Linux need no define, but they do need an `rpath`, or the executable will not find the library beside it at launch:

```
# macOS
cc host.c flaris-lib/lib/libflaris.dylib -Iflaris-lib/include \
    -Wl,-rpath,@executable_path -o host

# Linux
cc host.c flaris-lib/lib/libflaris.so -Iflaris-lib/include \
    -Wl,-rpath,'$ORIGIN' -o host
```

2. Hello, host

Complete and compilable — nothing is elided.

```
#include "flaris.h"
#include <stdio.h>

static const char *SCRIPT =
    "fn Greet(who: string): string {\n"
    "    return \"hello, \" + who;\n"
    "}\n";

int main(void)
{
    if (FlarisInit(NULL) != FLARIS_OK)
        return 1;

    if (FlarisLoadSource(SCRIPT, "greet.flr") != FLARIS_OK) {
        FlarisShutdown(1);
        return 1;
    }

    FlarisValue args[1] = { FlarisString("world") };
    FlarisValue result;
    char err[256];

    int rc = FlarisCallV("Greet", args, 1, &result, err, sizeof err);
    if (rc == FLARIS_OK) {
        size_t len = 0;
        const char *text = FlarisAsString(result, &len);
        printf("%.s\n", (int)len, text);
        FlarisReleaseValue(result);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(args[0]); /* FlarisCallV borrows; you still own it */
    FlarisShutdown(0);
    return 0;
}
```

```
$ cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -framework Security -framework CoreFoundation
$ ./host
hello, world
```

Three things in that program are worth naming now, because everything else builds on them.

`FlarisValue` is an **opaque handle** to a value living inside the VM. It is a single word; you never dereference it, and every read goes through a function like `FlarisAsString`. Nothing is copied when one crosses the

boundary.

`FlarisCallV` **borrow**s its arguments — you built them, you release them. The **result is yours**, and `FlarisReleaseValue` is how you drop it.

Every entry point returns a status code rather than aborting. A script that throws, a file that will not compile, a corrupt `.flx`, a module whose pinned hash does not match and a function that does not exist are all ordinary return values.

3. Shipping scripts with your application

You can hand Flaris either source (`.fls`) or compiled bytecode (`.flx`). For anything you ship, prefer bytecode:

- it does not expose your script source,
- it skips compilation at startup,
- it is validated on load, so a corrupt or mismatched file is refused rather than half-run.

Compile with the `flarism` toolchain as part of your build:

```
flarism --compile game.flx game.flx
```

Then load it at runtime:

```
FlarisLoadFile("game.flx");
```

A script you load this way is a **library of functions**, not a program. Give it either an `export` list or a `Main`, because `--compile` requires one of the two:

```
fn Greet(who: string): string { return "hello, " + who; }
fn Update(dt: float): int     { /* ... */ return 0; }

export { Greet, Update };
```

Everything the script declares is callable by name afterwards, whether or not it appears in the `export` list — the list is there to satisfy the compiler and to document intent.

Bytecode is portable across platforms and machines: a `.flx` built on one target runs on any other.

4. Lifecycle

```
FlarisInit(&cfg)
|
└─ FlarisLoadFile("game.flx")    ← or FlarisLoadSource(...)
```

```

|
├─ FlarisCallV(...)      ← as often as you like
├─ FlarisPump(0)         ← if the script uses timers/fibers/IO
|
FlarisShutdown(0)

```

`FlarisShutdown` releases everything and leaves your process able to start a fresh VM, so reloading a script means shutting down and initialising again. It is idempotent: calling it twice, or without a matching init, does nothing.

5. Configuration

Pass `NULL` to `FlarisInit` for defaults, or fill in a `FlarisConfig`:

```

FlarisConfig cfg = flarisConfigDefaults;
cfg.installSignals = FLARIS_OFF;      /* keep your own signal handlers */
cfg.startIoPool    = FLARIS_OFF;      /* stay single-threaded          */
cfg.unsafe         = FLARIS_ON;       /* the script may use Ffi/Memory */
cfg.maxFibers      = 64;               /* power of two                  */
cfg.stackSize      = 8192;
cfg.libsPath       = "/opt/myapp/scripts";

if (FlarisInit(&cfg) != FLARIS_OK) {
    /* a value was rejected – nothing was initialised */
}

```

Switches

Every switch is tri-state: `FLARIS_DEFAULT` (which is `0`, so a zeroed struct is a valid starting point), `FLARIS_ON`, or `FLARIS_OFF`. Anything else is rejected rather than read as a default.

Switch	Default	What it does
<code>installSignals</code>	on	Installs the VM's <code>SIGINT</code> / <code>SIGSEGV</code> handlers. Off leaves yours alone — a fault inside the VM then reaches <i>you</i> .
<code>startIoPool</code>	on	Starts the I/O worker threads. Off keeps your process single-threaded; I/O still completes, inline on the VM thread.
<code>unsafe</code>	off	Allows <code>Ffi.*</code> , raw <code>Memory</code> access and <code>Buffer</code> addresses. Off, those raise at runtime. This is the <code>--unsafe</code> flag.
<code>jit</code>	on	Native code generation.
<code>optimizations</code>	on	Constant folding and peephole passes.
<code>debugInfo</code>	on	Keeps line numbers and names, so stack traces are useful.
<code>signatureChecks</code>	on	Verifies a <code>.flx</code> fingerprint when one is pinned.
<code>requireSigned</code>	off	Refuses <code>.flx</code> not signed by a trusted key.
<code>verbose</code>	off	Emits the VM's own progress chatter at <code>FLARIS_LOG_INFO</code> .
<code>colors</code>	auto	ANSI colour in diagnostics. Irrelevant once you set a log handler.

unsafe is a trust decision. It lets a script load native code into your process via `Ffi` and read or write raw addresses via `Memory`. Leave it off for scripts you do not control.

Limits

Limit	Default	Range
<code>stackSize</code>	4096	65–65535 value-stack slots
<code>fifoSize</code>	1024	power of two, ≤ 1024
<code>maxSlabs</code>	1024	object-pool ceiling
<code>maxFibers</code>	256	power of two, ≤ 1024
<code>maxFrames</code>	64	8–1024 call frames per fiber
<code>ioThreads</code>	0	0 scales to the CPU count; ≤ 16
<code>libsPath</code>	none	Where <code>import</code> looks, <code>;</code> -separated. Borrowed — must outlive the VM.

Every limit reads `0` as "keep the default", so you set only what you care about. An out-of-range value makes `FlarisInit` **fail** rather than being quietly clamped, and nothing is initialised when it does.

6. Loading code

```
FlarisLoadFile("game.flx");           /* bytecode – the shipping path */
FlarisLoadFile("game.flc");           /* source, compiled on load      */
FlarisLoadSource(text, "<host>");      /* source from memory          */
FlarisRunFile("program.flc");         /* ... and call Main(), returning its code */
```

`FlarisLoadFile` chooses by extension: `.flx` is loaded as bytecode, anything else is compiled as source.

All three loaders run the script's **top level** — global initialisers and function definitions — and stop. They do not call `Main`, and `FlarisLoadSource` does not require the script to have one. `FlarisRunFile` is the exception: it is about to call `Main`, so it insists on one and returns its exit code instead of exiting your process.

Load one script per VM. To swap scripts, call `FlarisShutdown` and initialise again.

7. Calling into Flaris

There are three ways to call in, and they differ only in how you name the callee:

	when
<code>FlarisCallV(name, ...)</code>	a global, named by string
<code>FlarisCallValue(fn, ...)</code>	a value you hold — a resolved global, a callback the script gave you, a bound method
<code>FlarisNew(class, ...)</code>	construct an instance, running its constructor

All three **borrow** their arguments: you built them, you release them, and the same values may be passed to several calls. All three hand back an **owned** result you release with `FlarisReleaseValue`, or `NULL` to have it released for you.

```

FlarisValue args[2] = { FlarisString("player"), FlarisInt(3) };
FlarisValue result;
char err[256];

int rc = FlarisCallV("Damage", args, 2, &result, err, sizeof err);
if (rc == FLARIS_OK) {
    printf("hp now %lld\n", (long long)FlarisAsInt(result));
    FlarisReleaseValue(result);
}
FlarisReleaseValue(args[0]);
FlarisReleaseValue(args[1]); /* a small int allocates nothing; releasing it
                             is still correct and still free */

```

Return	Meaning
FLARIS_OK	result holds the return value
FLARIS_ERR_RAISED	the script threw; err holds "code: message"
FLARIS_ERR_NOTFOUND	no such function in the loaded script
FLARIS_ERR_NOT_FN	the name exists but is not a function
FLARIS_ERR_SUSPENDED	the function suspended (see below)
FLARIS_ERR_ARGS	bad argument count, or more than 32 arguments

`FlarisHasFunction("Damage")` reports whether a name is callable — useful when the script decides which hooks it implements:

```

if (FlarisHasFunction("OnPlayerJoin")) {
    FlarisValue a = FlarisString(playerName);
    FlarisCallV("OnPlayerJoin", &a, 1, NULL, err, sizeof err);
    FlarisReleaseValue(a);
}

```

Passing `NULL` for `outResult` is fine — the return value is released for you.

Native code is used. A function the JIT compiled (the `jit` switch, on by default) runs its native code when called this way; only the argument marshalling is interpreted, the same as for a call made from script code.

A called function must run to completion. If it yields or awaits, the call returns `FLARIS_ERR_SUSPENDED`, because nothing above it can resume it. For work that suspends, have the script start a fiber and drive `FlarisPump` instead.

8. Values and ownership

A `FlarisValue` is a one-word opaque handle to a value inside the VM. Do not dereference it, do not store it in a struct expecting to inspect it, and do not assume anything about its bit pattern. Everything you can do with one is a function call.

Making values

```
FlarisValue v;  
  
v = FlarisNil();  
v = FlarisBool(1);  
v = FlarisInt(42);  
v = FlarisFloat(3.5);  
v = FlarisString("text");           /* NUL-terminated */  
v = FlarisStringN(buf, len);        /* explicit length; may contain NULs */  
v = FlarisArrayNew(16);             /* capacity hint, starts empty */  
v = FlarisObjectNew();              /* string-keyed */
```

Each of these returns a value **you own**.

Reading values

Test first, then read. A read of the wrong type gives `0` or `NULL` rather than faulting, so a missing check misbehaves visibly instead of crashing.

```
if (FlarisIsInt(v))    int64_t n = FlarisAsInt(v);  
if (FlarisIsFloat(v)) double d = FlarisAsFloat(v);  
if (FlarisIsBool(v))  int      b = FlarisAsBool(v);  
  
if (FlarisIsString(v)) {  
    size_t len = 0;  
    const char *p = FlarisAsString(v, &len); /* borrowed, not a copy */  
    fwrite(p, 1, len, stdout);  
}
```

`FlarisAsString` hands back a pointer into the VM's own storage. It is valid while the value is, and it is not yours to free. Copy it if you need to keep it. It returns `NULL` for anything that is not a string — it will not coerce a number into text for you.

Arrays and objects

```
/* build */  
FlarisValue a = FlarisArrayNew(3);  
FlarisArrayAppend(a, FlarisInt(1)); /* append CONSUMES the value */  
FlarisArrayAppend(a, FlarisInt(2));  
  
FlarisValue o = FlarisObjectNew();  
FlarisObjectSet(o, "name", FlarisString("sensor")); /* set CONSUMES too */  
FlarisObjectSet(o, "id", FlarisInt(7));  
  
/* read */  
int n = FlarisArrayLen(a);
```

```
for (int i = 0; i < n; i++) {
    FlarisValue e = FlarisArrayGet(a, i);    /* OWNED - release it */
    process(FlarisAsInt(e));
    FlarisReleaseValue(e);
}

FlarisValue name = FlarisObjectGet(o, "name");    /* borrowed */
```

For numeric arrays, skip the handle entirely — these allocate nothing and need no release, which matters when you are walking thousands of elements:

```
int64_t x = FlarisArrayGetInt(a, i);
double y = FlarisArrayGetFloat(a, i);
```

And for bulk binary data, a `Block` hands you its bytes directly, so there is no per-element call at all:

```
size_t len = 0;
unsigned char *bytes = FlarisBlockData(v, &len);
```

The ownership rules

There are three, and they are the same rules in both directions.

1. What you make, you own. Every constructor above returns a value with one reference belonging to you. Release it with `FlarisReleaseValue` when you are done.

2. Some calls take it off your hands. `FlarisArrayAppend` and `FlarisObjectSet` **consume** the value you pass — the container takes your reference. Do not release it afterwards. Returning a value from a native module function (section 9) consumes it the same way: the VM takes it.

3. Borrowed values are not yours. Arguments handed to your native module function are **borrowed** — valid for that call only. If you need to keep one past the call, take your own reference:

```
FlarisValue kept = FlarisRetain(args[0]);
/* ... later ... */
FlarisReleaseValue(kept);
```

`FlarisObjectGet` also borrows. `FlarisArrayGet` does **not** — it returns an owned value, because a `[float]` array stores raw doubles and has no element object to lend you.

Scalars — `nil`, `bool`, small ints, chars — allocate nothing, so retaining and releasing them is free. You still write the calls; they simply cost nothing.

Resolving once

`FlarisCallV` hashes the name and walks the global environment on every call. For anything called repeatedly — a per-frame `Update`, a per-request handler — resolve it once and keep the value:

```
FlarisValue update = FlarisGetFunction("Update");    /* owned */

while (running) {
    FlarisValue dt = FlarisFloat(delta);
    FlarisCallValue(update, &dt, 1, NULL, err, sizeof err);
    FlarisReleaseValue(dt);
}

FlarisReleaseValue(update);
```

Measured on the boundary suite, that is 17.1 ns per call by name against 9.2 ns through a held value.

`FlarisIsCallable` tells you whether a value can be called as it stands. It is false for a method that still needs a receiver — see section 9 for `FlarisGetMethod`.

9. Native modules: your C functions, called from script

Publish C functions as a module and scripts call them by name, exactly as they call a built-in namespace. Nothing is marshalled at the boundary — your function receives the same handles the script is holding.

A complete example

```
#include "flaris.h"
#include <stdio.h>

/* — the functions scripts will call ————— */

static double gLastOutput;

/* Device.Read(pin) -> float */
static FlarisValue DeviceRead(FlarisValue *args, int argc)
{
    (void)argc;
    int pin = (int)FlarisAsInt(args[0]);
    return FlarisFloat(read_sensor(pin));
}

/* Device.SetOutput(channel, value) -> nil */
static FlarisValue DeviceSetOutput(FlarisValue *args, int argc)
{
    (void)argc;
    gLastOutput = FlarisAsFloat(args[1]);
    write_channel((int)FlarisAsInt(args[0]), gLastOutput);
    return FlarisNil();          /* a "void" function returns nil */
}
```

```

/* Device.Status() -> object */
static FlarisValue DeviceStatus(FlarisValue *args, int argc)
{
    (void)args; (void)argc;
    FlarisValue o = FlarisObjectNew();
    FlarisObjectSet(o, "output", FlarisFloat(gLastOutput));
    FlarisObjectSet(o, "online", FlarisBool(1));
    return o; /* the VM takes this reference */
}

/* — the table ————— */
/* name      function      returns      argument types      req opt flags */
static const FlarisNative DEVICE[] = {
    {"Read",      DeviceRead,      FLARIS_T_FLOAT, {FLARIS_T_INT},      1, 0,
FLARIS_FN_JIT_SAFE},
    {"SetOutput", DeviceSetOutput, FLARIS_T_NIL, {FLARIS_T_INT, FLARIS_T_FLOAT}, 2, 0, 0},
    {"Status",    DeviceStatus,    FLARIS_T_OBJECT, {0},      0, 0, 0},
};

/* — wiring ————— */

static const char *SCRIPT =
    "fn Sample(pin: int): float {\n"
    "    let v = Device.Read(pin);\n"
    "    Device.SetOutput(1, v * 2.0);\n"
    "    return v;\n"
    "}\n";

int main(void)
{
    if (FlarisInit(NULL) != FLARIS_OK)
        return 1;

    /* BEFORE loading: the compiler type-checks Device.* from this table. */
    if (FlarisRegisterModule("Device", DEVICE, 3) != FLARIS_OK) {
        FlarisShutdown(1);
        return 1;
    }

    if (FlarisLoadSource(SCRIPT, "control.flc") != FLARIS_OK) {
        FlarisShutdown(1);
        return 1;
    }

    FlarisValue pin = FlarisInt(4);
    FlarisValue out;
    char err[256];

```

```

    if (FlarisCallV("Sample", &pin, 1, &out, err, sizeof err) == FLARIS_OK) {
        printf("read %.2f\n", FlarisAsFloat(out));
        FlarisReleaseValue(out);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(pin);
    FlarisShutdown(0);
    return 0;
}

```

The script writes `Device.Read(pin)` the way it writes `Math.Abs(x)`. Because the compiler knows the signature you registered, passing a string where an int is declared is a **compile error**, not a runtime surprise.

Declaring the signature

`returnType` and each entry in `argTypes` is one of:

```

FLARIS_T_NIL      FLARIS_T_BOOL    FLARIS_T_INT      FLARIS_T_FLOAT
FLARIS_T_CHAR     FLARIS_T_STRING  FLARIS_T_ARRAY    FLARIS_T_OBJECT
FLARIS_T_BLOCK    FLARIS_T_ANY

```

They are bit flags, so `FLARIS_T_INT | FLARIS_T_FLOAT` accepts either. `0` in a slot means the same as `FLARIS_T_ANY`.

`required` is how many arguments must be supplied and `optional` how many more may be. A function declaring `2, 1` accepts two or three; check `argc` yourself to see which you got. The maximum is six.

Ownership inside a native function

Two rules, both from section 8:

- **args are borrowed.** Valid for the duration of the call. To keep one, take a reference with `FlarisRetain` and release it later.
- **Your return value is consumed.** The VM takes the reference. Return `FlarisNil()` if you have nothing to give back — never a value you also released.

Making the calls fast

By default, a script function containing a call into your module is **not JIT-compiled at all** — one call the compiler cannot make discards the native code for the whole enclosing function. `FLARIS_FN_JIT_SAFE` lifts that, and it is worth a great deal in a loop:

Device.Read(i) inside a hot loop	ns per call
without the flag	136
with <code>FLARIS_FN_JIT_SAFE</code>	7.8

It is a promise the VM cannot verify, so set it only when your function:

- does not yield, await or suspend;
- does not call back into Flaris;
- does not keep a borrowed argument past the call without `FlarisRetain`;
- does not assume an interpreter frame exists.

Raising an error and having side effects are both fine. `FLARIS_FN_PURE` is a stronger claim — that the result depends only on the arguments and the call has no observable effect — which additionally lets the optimiser hoist or drop the call. Do not set it on anything reading a device, a clock or a counter.

Flag the functions your hot paths call. Flagging one function in a module whose others are unflagged still leaves any script function calling the others interpreted.

Rules and limits

- **Register before you load.** The compiler types these calls from the registry and the loader binds them; a module registered afterwards does not reach a script that is already loaded.
- **Names are matched by hash.** Two names that collide in 32 bits cannot both be registered — `FlarisRegisterModule` returns `FLARIS_ERR_COLLISION` and you rename one. A name that does not resolve when a script loads rejects that script rather than failing later at the call.
- **A module name cannot shadow a built-in** (`Math`, `Json`, `File`, ...) — that returns `FLARIS_ERR_REGISTERED`.
- **Compiled scripts become yours.** A `.flx` containing calls into your module runs only in a process that registers those exact names. That is the intended shape: these are your application's scripts, compiled when your host loads them, not artifacts meant to run under a plain `flarism`.
- **No unsafe needed.** Unlike the FFI plugin path, a native module works with the default configuration.

10. Restricting what a script can reach

By default a script can use every built-in module: the filesystem, sockets, processes, everything. A host withholds whole modules with a bitmask, one bit per module:

```
FlarisConfig cfg = flarisConfigDefaults;
cfg.deniedModules = FLARIS_MOD_FILE | FLARIS_MOD_DIRECTORY |
                   FLARIS_MOD_FILEWATCH |
                   FLARIS_MOD_NET | FLARIS_MOD_OS |
                   FLARIS_MOD_FFI | FLARIS_MOD_BUFFER | FLARIS_MOD_MEMORY;
cfg.allowTls = FLARIS_OFF;
FlarisInit(&cfg);
```

A call into a denied module raises `Exception.UnsafeOperation`, which the script can catch and your `FlarisCallV` sees as `FLARIS_ERR_RAISED`. Nothing else about the script changes, and an allowed module costs nothing — the check is a bit test on an operand the call already carries.

The constants are `FLARIS_MOD_` plus the module name in upper case: `FLARIS_MOD_FILE`, `FLARIS_MOD_NET`, `FLARIS_MOD_OS`, `FLARIS_MOD_JSON` and so on, one for each of the 32 built-in modules. They are positional, so recompile against the header you ship with.

TLS is the one capability that is not a module — it is reached through `Net` and `Stream` — so it has its own switch, `cfg.allowTls`. Turning it off also refuses `https://` module imports.

What this is, and what it is not

It is **refusal, not confinement**. You can remove the filesystem; you cannot grant a subdirectory. There is no path allow-list, no host-supplied resolver and no per-call hook.

The unit is the **whole module**. Denying `FLARIS_MOD_OS` removes `Os.Execute` and `Os.Spawn`, and also `Os.Args` and `Os.Arch`, which only report the environment. It errs towards denying more.

And it bounds **capability, not consumption**. There is no interrupt, timeout or instruction budget: a script with every module denied can still loop forever and your `FlarisCallV` will not return. If you are running code you do not trust, that is the gap to plan around — run it in a process you can kill.

10a. Signals from the script

`Vm.NotifyHost(code)` hands a code to you, and does nothing if you are not listening:

```
static void OnNotify(int code, FlarisValue payload, void *userData)
{
    (void)userData;
    if (code == 7 && FlarisIsString(payload))
        SetStatusText(FlarisAsString(payload, NULL));
}

FlarisSetNotifyHandler(OnNotify, NULL);
```

```
fn Rebuild(items: [int]) {
    for (let i: int = 0; i < items.Length; i++) {
        Step(items[i]);
        VM.NotifyHost(7, "rebuilding " + str(i));
    }
}
```

The payload is optional and may be any value; it is **borrowed**, so retain it if you keep it. The call returns `true` when a handler ran and `false` when none is installed.

That last part is the point of it. A script calling `Vm.NotifyHost` still compiles and runs under a host that ignores it, and under the plain `flarism` command — where it simply returns `false`. A native module cannot do that: a script calling `Host.Progress(...)` will not compile unless the module exists. Use a native module when you want types and a real name; use this when you want a channel that is always there.

It is cooperative. It fires only where the script chose to call it, so it is not a way to stop a script that never returns. There is no such facility — see section 13.

10b. Embedding from C#

`libflaris` is a plain C shared library with 52 exported symbols, all prefixed `Flaris`, and no other public surface. That makes it a direct P/Invoke target — no C++ name mangling, no wrapper layer, no generated glue.

Everything in this guide works from C#, including native modules whose functions are C# methods.

The pieces that need care

FlarisValue is one word. Wrap it in a `readonly struct` over `nuint` so the compiler keeps handles apart from integers:

```
public readonly struct FlarisValue(nuint raw) { public readonly nuint Raw = raw; }
```

Use `[LibraryImport]`, not `[DllImport]`. It generates the marshalling at compile time, and `StringMarshalling.Utf8` gives you the UTF-8 the API expects:

```
[LibraryImport("flaris", StringMarshalling = StringMarshalling.Utf8)]
public static partial int FlarisLoadSource(string source, string name);
```

Callbacks must be `[UnmanagedCallersOnly]`, not delegates. A delegate needs the GC to keep it alive and cannot be called from JIT-compiled Flaris code. An `[UnmanagedCallersOnly]` static method gives you a real function pointer:

```
[UnmanagedCallersOnly]
static FlarisValue DeviceRead(FlarisValue* args, int argc)
    => FlarisFloat(FlarisAsInt(args[0]) * 1.5);
```

The two structs are sequential and blittable. `FlarisNative` holds a fixed-size array, so it needs `unsafe` and a `fixed` buffer:

```
[StructLayout(LayoutKind.Sequential)]
public unsafe struct FlarisNative
{
    public IntPtr Name; // 0
    public delegate* unmanaged<FlarisValue*, int, FlarisValue> Fn; // 8
    public uint ReturnType; // 16
    public fixed uint ArgTypes[6]; // 20
    public byte Required, Optional; // 44, 45
    public uint Flags; // 48
} // 56 bytes
```

`FlarisConfig` is 80 bytes and needs no attributes beyond `LayoutKind.Sequential` — every field is an `int`, `uint`, `ushort`, `ulong` or pointer, in declaration order.

Registering a module of C# functions

```
var fns = stackalloc FlarisNative[1];
fns[0].Name      = Marshal.StringToHGlobalAnsi("Read");
fns[0].Fn        = &DeviceRead;
fns[0].ReturnType = T.Float;
fns[0].ArgTypes[0] = T.Int;
fns[0].Required  = 1;
fns[0].Flags     = FlarisNative.JitSafe;

FlarisRegisterModule("Device", fns, 1);
FlarisLoadSource("fn Sample(p: int): float { return Device.Read(p); }", "s");
```

The script calls `Device.Read(p)` and your C# method runs. The name pointer must outlive registration — allocate it unmanaged, or pin it — but the VM copies the table itself, so `stackalloc` for the array is fine.

What is verified to work

A C# host built this way was checked against the library: initialising with a denied-module mask, registering a module of C# methods, calling scripts by name, strings and arrays round-tripping in both directions, reading an object's keys, constructing a script class from C# and calling a bound method on the instance, receiving `Vm.NotifyHost`, and seeing a script exception arrive as a return code rather than an unwind through native frames.

The constraint that matters most

The threading rule in section 13 applies unchanged: **every call must come from the thread that called `FlarisInit`**. In C# that means no `async` continuation that might resume elsewhere, and no `Task.Run` around a call. If your application is `async`, marshal Flaris work onto a single dedicated thread and keep it there.

Note also that a native module function running on the VM thread must not block for long: the scheduler is cooperative, so a C# method that waits stalls every fiber.

11. Driving the scheduler

Scripts that use timers, fibers or `async` I/O need scheduler time. You choose who owns the loop.

Your loop owns it. `FlarisPump` runs one non-blocking pass and returns how many fibers ran. It **never sleeps**, so your application decides its own cadence:

```
while (running) {
    FlarisPump(0);          /* 0 = every fiber that is ready right now */
    RenderFrame();
}
```

Bound the work per frame so a busy script cannot stall you:

```
FlarisPump(4);          /* at most four fibers this frame */
```

The VM owns it. `FlarisRunToCompletion` runs until nothing can become ready again, sleeping while fibers wait — what the `flarism` command does:

```
FlarisCallV("Start", NULL, 0, NULL, err, sizeof err);
FlarisRunToCompletion();
```

`FlarisHasWork()` is true while a queued fiber, a live timer, a claimed event or an in-flight I/O operation could still make progress — the termination condition for a loop of your own:

```
while (FlarisHasWork()) {
    if (FlarisPump(0) == 0)
        SleepMilliseconds(1);    /* nothing was ready – your call how to idle */
}
```

Because `FlarisPump` never sleeps, a loop with no idle of its own can spin thousands of times before a 1 ms timer comes due. That is deliberate: the VM does not decide how long your application blocks.

12. Errors and diagnostics

Script errors come back as values

A script that throws does not print and vanish. The call returns `FLARIS_ERR_RAISED` and writes the exception's code and message into your buffer:

```
char err[256];
int rc = FlarisCallV("Risky", NULL, 0, &result, err, sizeof err);

if (rc == FLARIS_ERR_RAISED)
    LogWarning("script error: %s", err);    /* e.g. "7: inventory full" */
```

`err` is always NUL-terminated and is cleared on entry, so a stale message never survives into a later call. The VM stays usable afterwards — the next call runs normally.

Passing `NULL` (with size `0`) for the buffer is supported and skips formatting the message entirely. `rc` still tells you the call raised, so a host that only branches on failure and never shows the text can use it:

```
int rc = FlarisCallV("Risky", NULL, 0, &result, NULL, 0); /* no message built */
```

That is worth doing only where raises are frequent enough to matter — building the message costs roughly a tenth of a microsecond per raised call. Where you report the error to a human or a log, take the buffer.

This holds for *any* failure inside the call, not just an explicit `throw`: a division by zero, an index out of bounds, or a blocked `Ffi` call with `unsafe` off all arrive the same way.

Nothing takes your process down

Nothing a script does, and nothing an input file contains, ends your process. An unhandled error is reported and unwound, a corrupt `.flx` is refused however deep the damage lies, a module whose pinned hash does not match is refused, a bundled dependency that fails the `requireSigned` check is withheld, and a module that fails to execute is refused — each as a return value, with the VM usable afterwards. That is what differs from the `flarism` command, which exits on all of these because exiting is the right thing for a command to do.

Two script-level constructs change meaning accordingly, because a script's decision to stop is not a decision that your application should stop:

In the script	Under flarism	Under your host
<code>Vm.Exit(code)</code>	exits with <code>code</code>	the call returns <code>FLARIS_ERR_RAISED</code> , naming the code
<code>Debug.AssertTrue / AssertEqual</code> failing exits		raises, after printing the trace

Two things are outside that guarantee, and both are yours to control:

- **Allocator exhaustion is fatal.** If the object pool hits `cfg.maxSlabs` or the system refuses memory, the VM reports and exits — object allocation has no failure return, so there is nothing to unwind to. Size `maxSlabs` for your workload; the default ceiling is 1024 slabs.
- **The signal handlers are opt-out.** With `installSignals` left on, the VM's `SIGSEGV`/`SIGINT` handlers end the process the way any such handler does. Set `cfg.installSignals = FLARIS_OFF` to keep your own.

Routing the VM's own output

By default the VM writes its diagnostics — runtime errors, warnings, compiler messages — to `stderr`. Hand it a callback instead:

```
static void MyLogger(int level, const char *message, void *userData)
{
    static const char *names[] = { "error", "warning", "info", "verbose" };
    fprintf(myLogFile, "[flaris %s] %s\n", names[level], message);
}

FlarisSetLogHandler(MyLogger, NULL);
```

`message` is formatted, NUL-terminated, and carries no trailing newline and no ANSI escapes; it is valid only for the duration of the call, so copy it if you keep it. Levels are `FLARIS_LOG_ERROR`, `FLARIS_LOG_WARNING`, `FLARIS_LOG_INFO` and `FLARIS_LOG_VERBOSE`. Pass `NULL` to go back to `stderr`.

You may set the handler before `FlarisInit`, so even startup diagnostics reach it. Compilation diagnostics go through it too, which is how you capture why a script failed to compile.

This covers what the *VM* says. What a *script* prints with `Console.WriteLine` still goes to `stdout`; redirect that by your own means if you need to.

13. Limits and what is not supported

One VM per process. VM state is process-wide, so there is no handle type and no way to hold two VMs at once. `FlarisShutdown` makes the process reusable, which covers reloading a script, but not isolating several scripts from each other.

One thread. Every function here must be called from the thread that called `FlarisInit`. Fibers are cooperative and run on that thread; the I/O worker threads never touch VM state.

One script per VM. Loading a second script into a live VM is not supported; shut down and initialise again.

A native module is bound to the host that registers it. Section 9 covers this in full: a `.flx` containing calls into your module runs only in a process that registers those exact names, and the standalone `flarism --compile` cannot compile a script that calls one, because it does not know your module. Compile such scripts from your own host, at load.

No resource limits. There is no way to interrupt a running script, cap its CPU time or bound its allocations. `FlarisCallV` runs the callee to completion. Section 10 covers what this means for untrusted code.

A malformed `.flx` is refused, not survived halfway. The bytecode reader abandons the whole load on the first inconsistency — `FlarisLoadFile` returns `FLARIS_ERR_COMPILE` and nothing from that file is defined. It does not attempt partial recovery, so a damaged file is never half-loaded.

An **FFI plugin** remains the alternative when the code you want to reach lives in a shared library you do not compile against: the script loads and calls it through `Ffi`, which requires `cfg.unsafe = FLARIS_ON` and is refused outright when `FLARIS_MOD_FFI` is denied. A native module needs neither.

14. API reference

Everything below is declared in `flaris.h`.

Lifecycle

Function	Description
<code>int FlarisInit(const FlarisConfig *cfg)</code>	Start the VM. <code>NULL</code> means defaults. <code>FLARIS_OK</code> or <code>FLARIS_ERR_INIT</code> .
<code>void FlarisShutdown(int code)</code>	Release everything; idempotent. <code>code</code> only reaches an attached debugger.
<code>void FlarisSetArgs(int argc, char **argv)</code>	Make <code>argv</code> visible to <code>0s.Args</code> . Borrows <code>argv</code> .
<code>void FlarisSetLogHandler(FlarisLogFn fn, void *userData)</code>	Route VM diagnostics to <code>fn</code> ; <code>NULL</code> restores <code>stderr</code> .

Loading

Function	Description
<code>int FlarisLoadSource(const char *source, const char *name)</code>	Compile a string and run its top level. <code>name</code> labels diagnostics.
<code>int FlarisLoadFile(const char *path)</code>	Load a <code>.flx</code> , or compile a <code>.fls</code> , and run its top level.
<code>int FlarisRunFile(const char *path)</code>	Load and call <code>Main()</code> , returning its exit code.

Calling

Function	Description
<code>int FlarisCallV(name, args, argc, outResult, errBuf, errSize)</code>	Call a global by name. Borrows <code>args</code> ; the result is owned.
<code>int FlarisCallValue(fn, args, argc, outResult, errBuf, errSize)</code>	Call a callable value you hold.
<code>int FlarisNew(className, args, argc, outResult, errBuf, errSize)</code>	Construct a class instance, running its constructor.
<code>FlarisValue FlarisGetFunction(const char *name)</code>	Resolve a global once. Owned — release it.
<code>FlarisValue FlarisGetMethod(obj, const char *name)</code>	A method bound to its receiver. Owned .
<code>int FlarisIsCallable(FlarisValue v)</code>	1 if the value can be called as it stands.
<code>int FlarisHasFunction(const char *name)</code>	1 if the name is callable.

Signals from the script

Function	Description
<code>void FlarisSetNotifyHandler(FlarisNotifyFn fn, void *userData)</code>	Listen for <code>Vm.NotifyHost(code[, payload])</code> . <code>NULL</code> stops listening.

Sandboxing

Field	Description
<code>uint64_t FlarisConfig.deniedModules</code>	Bitmask of <code>FLARIS_MOD_*</code> . 0 (the default) allows everything.
<code>int FlarisConfig.allowTls</code>	<code>FLARIS_OFF</code> refuses TLS connections, including <code>https://</code> imports.

Native modules

Function	Description
<code>int FlarisRegisterModule(name, fns, count)</code>	Publish C functions as the module <code>name</code> . Call before loading.
<code>FLARIS_FN_JIT_SAFE</code>	Flag: native code may call this function directly. Roughly 3x faster; see the promise above.
<code>FLARIS_FN_PURE</code>	Flag: no side effects, result depends only on the arguments. Implies JIT-safe.

Values

`FlarisValue` is an opaque handle to a value inside the VM. Constructors return an owned value; readers return borrowed data valid while the value is.

Function	Description
<code>FlarisNil()</code> / <code>FlarisBool(v)</code> / <code>FlarisInt(v)</code> / <code>FlarisFloat(v)</code>	Scalars.
<code>FlarisString(s)</code> / <code>FlarisStringN(s, len)</code>	Strings.
<code>FlarisIsNil(v)</code> / <code>FlarisIsBool</code> / <code>FlarisIsInt</code> / <code>FlarisIsFloat</code>	Scalar type tests.
<code>FlarisIsString(v)</code> / <code>FlarisIsArray(v)</code> / <code>FlarisIsObject(v)</code>	Container type tests.
<code>FlarisIsInstance(v)</code> / <code>FlarisClassName(v)</code>	A class instance, and its class name.
<code>FlarisAsInt(v)</code> / <code>FlarisAsFloat(v)</code> / <code>FlarisAsBool(v)</code>	Numeric reads; 0 on a type mismatch.
<code>const char *FlarisAsString(v, &len)</code>	Borrowed chars, <code>NULL</code> for a non-string.

Function	Description
<code>void *FlarisBlockData(v, &len)</code>	A Block's bytes directly — bulk data with no per-element call.
<code>FlarisArrayNew(cap) / FlarisArrayLen(a) / FlarisArrayAppend(a, v)</code>	Arrays; append consumes <code>v</code> .
<code>FlarisArrayGet(a, i)</code>	Owned element — release it.
<code>FlarisArrayGetInt(a, i) / FlarisArrayGetFloat(a, i)</code>	Numeric elements without boxing.
<code>FlarisObjectNew() / FlarisObjectLen(o) / FlarisObjectGet(o, key)</code>	Objects; <code>Get</code> borrows.
<code>FlarisObjectSet(o, key, v)</code>	Consumes <code>v</code> . Works on instances too.
<code>FlarisObjectNext(o, &cursor, &key, &keyLen, &value)</code>	Walk every property in insertion order. Both borrowed.
<code>FlarisRetain(v) / FlarisReleaseValue(v)</code>	Keep a borrowed value / drop an owned one.

Scheduler

Function	Description
<code>int FlarisPump(int maxFibers)</code>	One non-blocking pass; returns fibers run. Never sleeps.
<code>int FlarisHasWork(void)</code>	True while anything could still make progress.
<code>void FlarisRunToCompletion(void)</code>	Run until quiescent, sleeping as needed.

Status codes

Constant	Value	Meaning
<code>FLARIS_OK</code>	0	Success
<code>FLARIS_ERR_NO_VM</code>	-1	No VM is running
<code>FLARIS_ERR_NOT_FN</code>	-2	The name exists but is not callable
<code>FLARIS_ERR_ARGS</code>	-3	Bad argument count
<code>FLARIS_ERR_RAISED</code>	-4	The script threw
<code>FLARIS_ERR_SUSPENDED</code>	-5	The function yielded or awaited
<code>FLARIS_ERR_INIT</code>	-10	A configuration value was rejected
<code>FLARIS_ERR_COMPILE</code>	-11	Script did not compile
<code>FLARIS_ERR_NOTFOUND</code>	-12	No such function
<code>FLARIS_ERR_REGISTERED</code>	-13	That module name is already taken
<code>FLARIS_ERR_COLLISION</code>	-14	Two names share a 32-bit hash; rename one